

MIGRATION_PLAN.md — Didattica DiBEST: da PHP procedurale a OOP (pattern Strangler Fig)

Portale Didattica DiBEST · ramo `feature/refactoring-oop` · versione del 05/10/2026

Ramo di lavoro: `feature/refactoring-oop`. Documento di analisi e pianificazione: **nessun file sorgente è stato modificato**. Regola d'oro dell'intero percorso: **ogni step va in produzione da solo e il sito continua a funzionare come prima** (stessi URL, stesse pagine, stesso database, stesse email, stessi PDF).

STATO DI AVANZAMENTO

Step	Stato	Note
0 — Rete di sicurezza e inventario	fatto	<code>strumenti/inventario_oop.php</code> (funzioni procedurali/facciate per file, query nelle pagine, classi in <code>src/</code> , versione PHP ed estensioni)
1 — Composer, autoload, bootstrap	fatto	<code>composer.json</code> , <code>src/autoload.php</code> (autoload PSR-4 senza Composer, usato sul server), <code>src/bootstrap.php</code> agganciato in <code>functions.php</code> ; <code>PdfSemplice / TtfSemplice</code> → <code>App\Infrastructure\Pdf\Documento / TrueType</code> con alias
2 — Core	fatto	<code>App\Core\Database</code> , <code>Config</code> , <code>Container</code> , <code>App</code> (ponte per il legacy); <code>db_query/db_righe/db_riga/db_valore/db_esegui</code> sono facciate; 462 prove + 10 test PHPUnit verdi
3 — Infrastruttura condivisa	da fare	prossimo step

Decisioni prese durante l'implementazione (aggiornano le sezioni 1 e 2)

- **Aggancio in `functions.php`, non in `config.php`**: `functions.php` è incluso da tutte le pagine, dai cron e dalle prove automatiche (che non caricano `config.php`), quindi le classi sono disponibili ovunque. Il container si avvia appena esiste la connessione `$conn`.
- **Sul server non si carica `vendor/`**: `src/autoload.php` (15 righe) carica le classi `App\` da `src/` senza Composer. `vendor/` resta solo in sviluppo (PHPUnit, PHP-CS-Fixer) ed è escluso da git; se per errore finisse sul server è comunque bloccato dal `.htaccess`. Niente controllo di piattaforma di Composer (`platform-check: false`): nessun rischio di pagina bianca per la versione di PHP.
- **Compatibilità PHP 8.0** finché non si conosce la versione del server (Step 0 sul server: `php strumenti/inventario_oop.php`): niente `enum` né `readonly` per ora; con PHP ≥ 8.1 si introducono negli step dei moduli. PHPUnit 9.6 (compatibile con 8.0).
- **Errori silenziosi conservati**: le facciate `db_*` restituiscono `false`, `[]`, `null`, `-1` come prima (il sito usa `mysqli_report(OFF)`). Le eccezioni di dominio arrivano con i Service dei moduli.

Comandi di sviluppo

- `composer install` (una volta) · `composer test` (PHPUnit + prove del portale) · `composer cs` (stile PSR-12 su `src/` e `tests/`) · `composer inventario`.

- Sul server, dopo il caricamento: le stesse verifiche di sempre (`strumenti/prove_server.sh` , `strumenti/verifica_sito.sh`).

0. FOTOGRAFIA DELLO STATO ATTUALE

Elemento	Situazione	Impatto sulla migrazione
Dimensioni	≈ 27.000 righe in 100 pagine PHP (52 pubbliche, 48 in <code>admin/</code>) + 11.300 righe in 27 file <code>inc/</code> (484 funzioni)	Troppo grande per una riscrittura: serve sostituzione incrementale
Bootstrap	<code>config.php</code> (sessione, header di sicurezza, lettura <code>.env</code> , connessione <code>\$conn</code>), <code>functions.php</code> (carica <code>inc/*.php</code> in ordine fisso, poi <code>assicura_schema()</code>), <code>middleware.php</code> (SSO, <code>\$user_info</code> , ruoli)	Punto unico in cui agganciare l'autoloader senza toccare le pagine
Database	<code>mysqli</code> con <code>mysqli_report(OFF)</code> ; 535 chiamate <code>->query()</code> , 246 <code>->prepare()</code> , 175 chiamate agli helper <code>db_query/db_righe/db_riga/db_valore/db_esegui</code> (<code>inc/base.php</code>)	Gli helper sono già un "mini-repository": diventano la prima facciata verso la classe <code>Database</code>
Tabelle	circa 56 tabelle (<code>database/schema.sql</code> + quelle create da <code>inc/schema.php</code>)	Ogni aggregato ha già tabelle ben riconoscibili: i Repository si ricavano direttamente
Stato globale	<code>\$conn</code> passato quasi sempre come primo parametro (solo 6 global in tutto il codice); <code>\$user_info</code> , <code>\$u_id</code> , <code>\$is_full_admin</code> , <code>\$utente_admin</code> , <code>\$filtro_p</code> creati dal <code>middleware</code> / dagli header	Situazione favorevole: la Dependency Injection si può introdurre senza caccia ai <code>global</code>
Logica e HTML	Pagine "a scheda": in alto le azioni POST (redirect + exit), sotto l'HTML; molte funzioni <code>inc/</code> restituiscono HTML (<code>html_*</code>)	Separare per gradi: prima query (Repository), poi regole (Service), per ultimo le viste
Schema	<code>inc/schema.php</code> → <code>assicura_schema()</code> con marcatore <code>cache/schema_vNN.ok</code> (oggi v43)	Va mantenuto; più avanti diventa un sistema di migrazioni a classi
Prove	<code>strumenti/prove/esegui.php</code> : 450 prove (funzioni su DB usa-e-getta + pagine dell'ambiente locale); <code>strumenti/prove_server.sh</code> , <code>strumenti/verifica_sito.sh</code>	È la rete di sicurezza: ogni step deve lasciarle tutte verdi
Rilascio	Upload manuale dei file su XAMPP (<code>/opt/lampp</code>), nessun Composer sul server, nessuna CI	L'autoload deve funzionare senza eseguire Composer in produzione
Classi esistenti	Solo <code>PdfSemplice</code> e <code>TtfSemplice</code> (<code>inc/pdf.php</code>)	Primi candidati naturali a spostarsi in <code>src/</code>

Moduli funzionali (dai file `inc/` e dalle schede `admin/`):

Area	File procedurali attuali
Fondamenta	<code>base</code> (utenti, SSO, permessi, CSRF, rate limit, email, helper DB), <code>sistema</code> , <code>dati</code> (log, audit, backup), <code>schema</code> , <code>esporta</code> , <code>pdf</code>
Portale	<code>sezioni</code> , <code>aspetto</code> , <code>organizzazione</code> (testata, menu, home, widget)
Eventi	<code>eventi_progetti</code> , <code>agenda</code> , <code>seminari</code> , <code>avvisi</code> , <code>report_ambiti</code>
Iscrizioni	<code>prenotazioni</code> , <code>liste_attesa</code> , <code>attestati</code>
Risorse e aule	<code>risorse</code> , <code>calendario_risorse</code>

Area	File procedurali attuali
Anagrafi	anagrafi , catalogo_ateneo
FSL	fsl (scuole, convenzioni, valutazioni)
Tutorato	tutorato , tutorato_registro
Didattica	didattica (moduli, pratiche, uffici, iter), convalide (protocollo, PDF/A, segreteria), sedute (consigli, verbali, firme)

1. SETUP PSR-4 E COMPOSER

1.1 Tecnologia scelta

- **PHP ≥ 8.1** (enum, proprietà `readonly` , `match` , tipi union). Primo controllo dello Step 0: versione PHP di `/opt/lampp/bin/php` . Se il server fosse fermo a 8.0 si rinuncia a enum/readonly finché non si aggiorna XAMPP, il resto del piano non cambia.
- **Composer** solo sulle macchine di sviluppo. In produzione non serve eseguirlo: si carica la cartella `vendor/` generata (inizialmente contiene **solo** l'autoloader di Composer, zero librerie).
- **Nessun framework** (Laravel/Symfony): imporrebbero una riscrittura e un cambio di modello di rilascio. Si adottano i loro principi (PSR-4, DI, Repository/Service) con codice nostro, piccolo e leggibile.
- Strumenti di qualità **solo in sviluppo** (`require-dev` , mai caricati sul server): PHPUnit (test di `src/`), PHPStan (analisi statica, livello crescente), PHP-CS-Fixer (stile PSR-12 solo su `src/` e `tests/`).

1.2 Struttura delle cartelle

```

/                                     (pagine legacy: restano dove sono, stessi URL)
├── composer.json
├── composer.lock
├── vendor/                          (generato: autoload ottimizzato; caricato sul server; protetto da .htaccess)
├── src/                             (codice OOP, namespace App\ ; protetto da .htaccess)
│   ├── bootstrap.php               (crea il Container e registra i servizi; incluso da config.php)
│   ├── Core/                      (Config, Database, Container, Session, Clock, Http\Request, Http\Response, Csrf)
│   ├── Infrastructure/             (Mail, Audit, Storage, Pdf, Docx, Xlsx, Logger)
│   ├── Auth/                      (Utente corrente, sincronizzazione SSO, politiche di accesso)
│   ├── Portale/                   (Testata, Menu, Home, Organizzazione)
│   ├── Eventi/                    (Area, Evento, Turno, Agenda, Seminari, Avvisi, Report)
│   ├── Iscrizioni/                (Prenotazione, ListaAttesa, Attestato)
│   ├── Risorse/                   (Risorsa, PrenotazioneRisorsa, Calendario)
│   ├── Anagrafi/                  (Persona, Docente, CorsoStudio, Insegnamento, CatalogoAteneo, Scuola)
│   ├── Fsl/                       (Convenzione, Valutazione)
│   ├── Tutorato/                  (Incarico, Registro, Firme)
│   ├── Didattica/                 (Moduli, Pratiche, Uffici, Iter, Protocollo, Convalide, Sedute, Consigli)
│   └── Legacy/                    (adattatori temporanei verso funzioni non ancora migrate)
├── tests/                          (PHPUnit: Unit/ e Integration/; protetto da .htaccess)
├── inc/                            (procedurale: si svuota modulo per modulo, le funzioni diventano facciate)
└── strumenti/prove/esegui.php      (prove esistenti: restano e crescono)

```

1.3 composer.json (da creare allo Step 1)

```
{
  "name": "dibest/didattica",
  "description": "Portale Didattica DiBEST",
  "type": "project",
  "require": {
    "php": ">=8.1",
    "ext-mysqli": "*",
    "ext-mbstring": "*",
    "ext-json": "*"
  },
  "require-dev": {
    "phpunit/phpunit": "^10.5",
    "phpstan/phpstan": "^1.11",
    "friendsofphp/php-cs-fixer": "^3.50"
  },
  "autoload": {
    "psr-4": { "App\\": "src/" }
  },
  "autoload-dev": {
    "psr-4": { "Tests\\": "tests/" }
  },
  "config": {
    "optimize-autoloader": true,
    "platform": { "php": "8.1.0" },
    "sort-packages": true
  },
  "scripts": {
    "test": ["phpunit", "php strumenti/prove/esegui.php"],
    "stan": "phpstan analyse src --level=5",
    "cs": "php-cs-fixer fix src tests --rules=@PSR12",
    "build": "composer install --no-dev --classmap-authoritative"
  }
}
```

- `platform.php` blocca le dipendenze alla versione del server, così ciò che funziona in sviluppo funziona anche lì.
- Prima di ogni rilascio: `composer build` → `vendor/` contiene solo l'autoload (nessuna libreria di sviluppo). `vendor/` **va versionato** sul ramo di rilascio (o caricato insieme ai file), perché sul server Composer non c'è.

1.4 Aggancio senza rompere nulla

Una sola riga in `config.php`, subito dopo la creazione di `$conn`:

```
// Autoload delle classi App\ (src/) e servizi condivisi; se manca vendor/ il sito continua in modalità solo procedurale
if (is_file(__DIR__ . '/vendor/autoload.php')) require_once __DIR__ . '/src/bootstrap.php';
```

`src/bootstrap.php` include `vendor/autoload.php`, crea il Container e registra la Database **attorno alla stessa `$conn`** (nessuna seconda connessione). Se il file manca, il sito si comporta esattamente come oggi: è il paracadute del primo rilascio.

1.5 Sicurezza delle nuove cartelle

`src/`, `vendor/`, `tests/` non devono mai essere raggiungibili dal web (come oggi `inc/`): in ognuna un `.htaccess` con `Require all denied`, più `composer.json` / `composer.lock` aggiunti al blocco `FilesMatch` del `.htaccess` principale. Le prove esistenti verificano già i 403 su `inc/`: si estendono alle nuove cartelle.

1.6 Convenzioni di codice

- `declare(strict_types=1);` in ogni file di `src/`.
- **Linguaggio di dominio in italiano** (come il codice e le tabelle attuali: `Pratica`, `Seduta`, `Ufficio`, metodi `trovaPerId`, `assegna`), **ruoli tecnici in inglese** (`Repository`, `Service`, `Controller`) per restare riconoscibili a chiunque conosca i pattern.
- Una classe per file, nome file = nome classe, PSR-12.
- Nessun `echo` / `header()` / `exit` dentro `Repository` e `Service`: solo valori di ritorno ed eccezioni di dominio.
- Le classi non leggono `$_GET`, `$_POST`, `$_SESSION`, `$_SERVER` direttamente: li ricevono (`Request`, `Session`, `CurrentUser`).

2. CORE & DATABASE ISOLATION

2.1 Obiettivo

Un'unica classe `App\Core\Database` iniettata nei costruttori dei `Repository` al posto di `$conn` passato a mano o letto con `global`. Le funzioni procedurali continuano a ricevere `$conn` finché non vengono migrate.

2.2 Classe `App\Core\Database` (sopra `mysqli`, non `PDO`)

Si resta su **mysqli** nella prima fase: stesso driver, stessi tipi restituiti, stesso `SET time_zone` e charset, stesso comportamento di `mysqli_report(OFF)`. Passare a `PDO` in un secondo momento sarà un cambio interno alla classe, invisibile al resto del codice.

```
namespace App\Core;

final class Database
{
    public function __construct(private \mysqli $conn) {}

    /** @return list<array<string,mixed>> */
    public function righe(string $sql, array $parametri = []): array;    // = db_righe()
    public function riga(string $sql, array $parametri = []): ?array;    // = db_riga()
    public function valore(string $sql, array $parametri = []): mixed;    // = db_valore()
    public function esegui(string $sql, array $parametri = []): int;    // = db_esegui(): righe toccate
    public function inserisci(string $sql, array $parametri = []): int;    // restituisce l'id inserito,
    letto SUBITO
    public function transazione(callable $lavoro): mixed;    // begin/commit/rollback (FOR
    UPDATE delle prenotazioni)
    public function mysqli(): \mysqli;    // uscita di emergenza per il
    codice legacy
}
```

Note dalla codebase: - `inserisci()` risolve il problema noto per cui `$conn->insert_id` viene azzerato da un `UPDATE` successivo (oggi si ripiega su `SELECT MAX(id)`). - Tipi dei parametri ricavati come oggi negli helper (`i/d/s`, `null` gestito). - Errori: oggi la maggior parte delle query fallisce in silenzio (`@$conn->query`). La classe registra sempre l'errore in `error_log` con la query (senza i valori dei parametri) e lancia `DatabaseException` **solo** nei metodi nuovi; le facciate legacy mantengono il comportamento silenzioso finché il chiamante non è migrato.

2.3 Facciate: gli helper esistenti delegano alla classe

In `inc/base.php` gli helper restano con la stessa firma, ma il corpo diventa:

```
function db_righe($conn, string $sql, array $par = []): array {  
    return \App\Core\App::db($conn)->righe($sql, $par);  
}
```

Così 175 punti del codice passano subito dalla nuova classe senza essere toccati, e la classe viene collaudata dalle 450 prove esistenti.

2.4 Container (Dependency Injection)

- `App\Core\Container`: container minimale scritto in casa (registrazione `set(id, factory)`, `get(id)` con istanze condivise, autowiring per costruttori con classi tipizzate). Circa 80 righe, nessuna dipendenza. Se in futuro servisse di più si sostituisce con `php-di/php-di` senza cambiare le classi, perché queste ricevono le dipendenze nel costruttore e non conoscono il container.
- `src/bootstrap.php` registra: `Config`, `Database` (sulla `$conn` di `config.php`), `Session`, `Clock`, `Mailer`, `AuditLog`, `Storage`, e poi `Repository` e `Service` man mano che nascono.
- `App\Core\App` è un **ponte statico usato solo dal codice legacy** (`App::get(PraticaService::class)`, `App::db($conn)`). Le classi in `src/` non lo usano mai: ricevono tutto dal costruttore. Quando l'ultima facciata sparisce, sparisce anche il ponte.

2.5 Sostituzione delle variabili globali e superglobali

Oggi	Domani	Come si arriva
<code>\$conn</code> (in <code>config.php</code> , parametro di quasi tutte le funzioni)	<code>Database</code> iniettata	Facciate (2.3); le nuove classi non vedono più <code>\$conn</code>
<code>.env</code> letto in <code>config.php</code> , costanti sparse (<code>URL_SITO</code> , <code>POSTI_SENZA_LIMITE</code> , <code>DIR_*</code> ...)	<code>App\Core\Config</code> (sola lettura)	Si legge lo stesso file; le costanti restano definite per il legacy e <code>Config</code> le espone tipizzate
<code>\$SESSION['utente_id']</code> , <code>utente_ruolo_id</code> , <code>auth_meta</code> ...	<code>App\Core\Session</code> + <code>App\Auth\UtenteCorrente</code>	<code>middleware.php</code> costruisce <code>UtenteCorrente</code> e continua a creare <code>\$user_info</code> , <code>\$u_id</code> , <code>\$is_full_admin</code> per le pagine non migrate
<code>\$_GET/\$_POST/\$_FILES/\$_SERVER</code> dentro funzioni (<code>leggi_risposte_modulo</code> , <code>crea_pratica</code> legge IP e sessione)	<code>App\Core\Http\Request</code> passata ai Controller; i Service ricevono dati già validati (DTO)	Modulo per modulo
<code>date()</code> , <code>time()</code> sparsi (scadenze, promemoria, conservazione dati)	<code>App\Core\Clock</code>	Permette test deterministici di promemoria e scadenze
<code>inviaNotificaEmail()</code> chiamata ovunque	<code>App\Infrastructure\Mail\Mailer</code> (interfaccia)	La funzione resta e delega; nei test un <code>MailerFinto</code> (le prove oggi la ridefiniscono)
<code>static \$cache</code> dentro le funzioni (<code>uffici_didattica</code> , <code>scelte_anagrafe_didattica</code> ...)	Cache nei Repository, con <code>svuota()</code> esplicito	Elimina i casi di dati "vecchi" come quello emerso sugli uffici appena creati

3. DOMAIN DRIVEN DESIGN

3.1 Strati e responsabilità

Pagina legacy / Controller → Service (regole, transazioni, eventi) → Repository (SQL) → Database
↓ ↑
Infrastructure (Mail, Pdf, Storage, Audit) Model / Value Object

- **Model (Entity):** oggetto con identità e stato del dominio (`Pratica` , `Seduta`). Costruito dal Repository (`daRiga(array)`), espone metodi di dominio (`$pratica->eConclusa()` , `$pratica->richiedeBollo()`), non contiene SQL.
- **Value Object:** immutabili, confrontati per valore: `CodiceFiscale` , `Email` , `AnnoAccademico` (`"2025/2026"`), `Protocollo` (numero + data), `Cfu` , `IntervalloDate` .
- **Enum** (PHP 8.1) al posto delle costanti array: `StatoPratica` (oggi `STATI_PRATICA`), `EsitoSeduta` , `EsitoConvalida` , `TipoUfficio` (`protocollo` , `cdl` , `segreteria`), `StatoPrenotazione` , `MetodoAccesso` (`SPID` , `CIE` , `Ateneo`), `StatoVerbale` . Ogni enum offre `etichetta()` e `colore()` per le viste: le costanti attuali restano come alias finché servono.
- **Repository:** tutto e solo l'SQL di un aggregato. Restituiscono Model/array tipizzati, accettano criteri espliciti (`cercaPerFiltri(FiltriPratiche $f)`). Uno per aggregato.
- **Service (Application Service):** un caso d'uso per metodo (`registraProtocollo` , `inviaAllaSegreteria`), apre la transazione, coordina Repository e Infrastructure, scrive l'audit, lancia eccezioni di dominio (`PraticaNonTrovata` , `ProtocolloNonValido`). È l'unico posto delle regole di business.
- **Domain Service / Policy:** regole riusabili senza stato: `PoliticaAccessoPratica` (oggi `utente_vede_pratica`), `CalcoloPostiDisponibili` (oggi `getPostiOccupati` , definita in `config.php` , + soglia `POSTI_SENZA_LIMITE`), `RisolutoreIter` (oggi `iter_modulo` CON `@cdl` / `@segreteria`).
- **Presenter/View helper:** le funzioni `html_*` diventano classi di presentazione in `src/<Modulo>/Vista/` (stesso HTML, prima) e solo alla fine, eventualmente, template.

3.2 Entità principali per contesto (bounded context)

Contesto	Entità / aggregati	Tabelle principali	Repository	Service
Auth & Utenti	<code>Utente</code> , <code>Ruolo</code> , <code>Abilitazione</code> , <code>UtenteCorrente</code> , <code>MetadatiAccesso</code>	<code>utenti</code> , <code>ruoli</code> , <code>abilitazioni_ambito</code> , <code>abilitazioni_attesa</code> , <code>log_accessi</code> , <code>rate_limit_attempts</code>	<code>UtenteRepository</code> , <code>AbilitazioneRepository</code>	<code>SincronizzazioneSso</code> , <code>ServizioPermessi</code> (oggi <code>ha_modulo</code> , <code>utente_ha_abilitazioni</code> , <code>utente_gestisce_didattica</code>)
Portale	<code>VoceMenu</code> , <code>Slide</code> , <code>ConfigurazionePortale</code> , <code>OrganizzazioneHome</code>	<code>menu_voci</code> , <code>slide_home</code> , <code>configurazione_portale</code> , <code>copie_configurazione</code> , <code>impostazioni_sistema</code>	<code>MenuRepository</code> , <code>ConfigurazioneRepository</code>	<code>ServizioOrganizzazione</code> (proposta, backup, ripristino)
Eventi	<code>Area</code> (<code>pagine_eventi</code>), <code>Evento</code> , <code>Turno</code> , <code>Seminario</code> , <code>IscrizioneAvvisi</code>	<code>pagine_eventi</code> , <code>eventi</code> , <code>turni</code> , <code>progetti_dettagli</code> , <code>sottocategorie</code> , <code>avvisi_eventi</code> , <code>avvisi_iscrizioni</code>	<code>AreaRepository</code> , <code>EventoRepository</code> , <code>TurnoRepository</code> , <code>AvvisoRepository</code>	<code>ServizioAgenda</code> , <code>ServizioAvvisi</code> , <code>ReportAmbiti</code>

Contesto	Entità / aggregati	Tabelle principali	Repository	Service
Iscrizioni	Prenotazione , ListaAttesa , Attestato , Sondaggio	prenotazioni (stati, lista d'attesa, presenze e attestati), partecipanti_prenotazione , messaggi_prenotazioni , campi_form , sondaggi , sondaggi_domande , sondaggi_risposte	PrenotazioneRepository , AttestatoRepository	ServizioPrenotazioni (con FOR UPDATE anti-overbooking), ServizioListaAttesa , ServizioAttestati , ServizioSondaggi
Risorse	Risorsa , PrenotazioneRisorsa , Sportello	risorse , risorse_orari , risorse_chiusure , prenotazioni_risorse	RisorsaRepository	ServizioCalendarioRisorse
Anagrafi	Persona , Docente , CorsoStudio , Insegnamento , Scuola , CorsoCatalogo	personale_ateneo , personale_modifiche , anagrafe_strutture , corsi_studio , insegnamenti , scuole , ateneo_cds , ateneo_insegnamenti , ateneo_insegnamenti_caricati	PersonaRepository , CorsoRepository , InsegnamentoRepository , CatalogoAteneoRepository	SincronizzazioneAnagrafe , ClientApiAteneo
FSL	Convenzione , ConvenzioneCompilata , ValutazioneFsl	convenzioni_scuole , convenzioni_compilate , valutazioni_fsl	ConvenzioneRepository	ServizioConvenzioni (documenti Word, PAdES)
Tutorato	Bando , Incarico , AttivitaRegistro , Firma	tutorato_bandi , tutorato_incarichi , tutorato_eventi , tutorato_registro	IncaricoRepository , RegistroRepository	ServizioIncarichi , ServizioFirmePades , ServizioRegistro
Didattica - Pratiche	Modulo (con Campo , ColonnaTabella , Condizione), Pratica , EventoPratica , Risposta , DecisioniConvalida , RigaConvalida	didattica_moduli , pratiche , pratiche_eventi , pratiche_operatori	ModuloRepository , PraticaRepository , EventoPraticaRepository	ServizioInvioPratica , ServizioIter , ServizioProtocollo , ServizioConvalide , ServizioSegreteria , GeneratoreDomandaPdfa
Didattica - Uffici	Ufficio (con TipoUfficio), Operatore	didattica_uffici , ufficio_didattica	UfficioRepository , OperatoreRepository	ServizioSmistamento
Didattica - Organi	Consiglio , ComponenteConsiglio , Seduta , Presenza , Convocazione , Verbale	didattica_consigli , didattica_consigli_personale , didattica_sedute , didattica_sedute_presenze , didattica_convocazioni	ConsiglioRepository , SedutaRepository	ServizioSedute , ServizioVerbale (Word, PAdES), ServizioEstratti

Contesto	Entità / aggregati	Tabelle principali	Repository	Service
Infrastruttura	—	log_email , log_attivita , template_email	—	Mailer , AuditLog , Storage (uploads protetti), Pdf (oggi PdfSemplice PDF/A), Docx , Xlsx , Logger

Esempio di confine dai casi recenti: oggi `registra_protocollo_pratica()` legge la pratica, aggiorna protocollo e bollo, scrive due eventi, risolve l'ufficio del corso, assegna e manda email. Domani:

```
final class ServizioProtocollo
{
    public function __construct(
        private PraticaRepository $pratiche,
        private ModuloRepository $moduli,
        private EventoPraticaRepository $eventi,
        private RisolutoreIter $iter,
        private ServizioSmistamento $smistamento,
        private Clock $orologio,
    ) {}

    public function registra(int $praticaId, Protocollo $protocollo, bool $bolloVerificato, Autore
    $autore): EsitoProtocollo
    {
        // regole: data non futura, bollo obbligatorio se il modulo lo richiede, passo successivo
        dell'iter
    }
}
```

e la funzione legacy diventa una facciata di 3 righe che converte parametri e risultato nel formato di oggi `[errore|null, messaggio]`.

3.3 Regole che restano invariate

- Lo schema del database non cambia per effetto del refactoring (le tabelle si rinominano solo se serve davvero, mai insieme a una migrazione di codice).
- Gli URL non cambiano. Le pagine diventano "controller sottili" restando file `.php` agli stessi indirizzi; un front controller con router è facoltativo e solo alla fine (Step 15).
- I testi visibili all'utente, le email e i PDF devono risultare identici prima e dopo ogni step (vedi prove di caratterizzazione).

4. ROADMAP A STEP INCREMENTALI

Come si lavora in ogni step (vale per tutti)

1. **Prove di caratterizzazione prima di toccare il codice:** si fissano in `strumenti/prove/esegui.php` (o in `tests/Integration`) gli output attuali delle funzioni del modulo: righe del DB, HTML significativo, email intercettate, PDF (testo estratto e conformità veraPDF dove serve).
2. **Nuove classi in `src/` + test unitari PHPUnit.**
3. **Le funzioni legacy diventano facciate** con la stessa firma e lo stesso risultato. Nessuna pagina viene toccata in questa fase.

4. **Pagine/schede del modulo** convertite in controller sottili (azioni POST → Service; vista che usa Repository/Presenter).
5. **Verifica:** `composer test` (PHPUnit + 450 prove esistenti), PHPStan sul nuovo codice, giro manuale delle pagine del modulo in locale (`strumenti/locale`).
6. **Rilascio:** un PR dal ramo dello step verso `main` , elenco esatto dei file da caricare, `prove_server.sh` e `verifica_sito.sh` dopo l'upload.
7. **Rollback:** ricaricare i file della versione precedente (le facciate fanno sì che nessuno step richieda modifiche al database; se uno step le richiede, sono solo additive e retrocompatibili).
8. Le facciate si eliminano **solo** quando nessun file le chiama più (verifica con `grep` e PHPStan), in uno step di pulizia separato.

Ramificazione: `feature/refactoring-oop` è il ramo di coordinamento (questo piano, `composer.json` , Core). Ogni step lavora su `feature/refactoring-oop/step-NN-nome` e va in `main` (e quindi in produzione) appena verde, così la distanza dal codice in produzione resta piccola e le nuove funzionalità continuano a entrare normalmente.

STEP 0 — Rete di sicurezza e inventario (nessun cambio in produzione)

- Verifica della versione PHP ed estensioni sul server (`prove_server.sh` già le elenca); decisione 8.1+ o 8.0.
- Inventario automatico delle funzioni `inc/` e dei loro chiamanti (script in `strumenti/`) per misurare l'avanzamento: "funzioni procedurali rimaste / facciate / migrate".
- Prove di caratterizzazione aggiuntive sulle parti meno coperte: prenotazione completa con lista d'attesa, generazione attestati, menu e home, esportazioni Excel/Word.
- **Fine step:** piano approvato, prove verdi, nessun file di produzione modificato.

STEP 1 — Composer, autoload PSR-4 e bootstrap (le fondamenta, zero cambi di comportamento)

- `composer.json` , `vendor/` generato con `composer build` , `src/bootstrap.php` , `.htaccess` di protezione per `src/` , `vendor/` , `tests/` .
- Riga di aggancio condizionale in `config.php` (1.4).
- Prima classe spostata come prova del meccanismo: `PdfSemplice` / `TtfSemplice` → `App\Infrastructure\Pdf\Documento` e `\TrueType` , con `class_alias` in `inc/pdf.php` per i nomi vecchi.
- Prove: 403 su `src/` , `vendor/` , `tests/` , `composer.json` ; PDF generati identici (stesse pagine, veraPDF conforme).
- **Rilascio:** `composer.json` , `composer.lock` , `vendor/` , `src/` , `.htaccess` nuovi, `config.php` , `inc/pdf.php` .

STEP 2 — Core: Config, Database, Container, Session, Clock

- Classi del paragrafo 2; helper `db_*` di `inc/base.php` trasformati in facciate.
- `Config` legge lo stesso `.env` / `.env.locale` con la stessa logica di `config.php` (che continua a funzionare per il legacy).
- Le prove esistenti girano sopra la nuova `Database` senza modifiche: è la verifica principale.
- **Rilascio:** `src/Core/*` , `src/bootstrap.php` , `inc/base.php` .

STEP 3 — Infrastruttura condivisa: Mailer, AuditLog, Storage, Logger, documenti

- `inviaNotificaEmail()` → Mailer (client SMTP attuale spostato in classe, stesso registro `log_email`); `registra_log_audit()` (`inc/sistema.php`, tabella `log_attivita`) → AuditLog; upload protetti (`secure_upload`, cartelle con `.htaccess` automatico, `allegato_pratica.php`) → Storage; Word/Excel (`docx_*`, `xlsx_crea`) → Infrastructure\Documenti.
- Tutte le funzioni restano come facciate: nessuna pagina cambia.
- **Perché qui:** ogni modulo successivo ne ha bisogno; migrarli prima evita di toccare due volte gli stessi file.

STEP 4 — Modulo pilota: Avvisi (periferico, piccolo, completo)

- `inc/avvisi.php` (7 funzioni) + `avvisi.php`: `AvvisoRepository`, `ServizioAvvisi`, enum degli ambiti, controller sottile per la pagina.
- È il **modello di riferimento** per tutti gli step successivi: struttura delle cartelle, test, facciate, controller, elenco file di rilascio. Si sceglie un modulo periferico come pilota proprio perché un errore qui non tocca login, prenotazioni o pratiche.
- **Fine step:** pagina avvisi identica, email di avviso identiche, documento "come migrare un modulo" aggiunto in fondo a questo piano con le lezioni apprese.

STEP 5 — Autenticazione, utente corrente e permessi

- `UtenteCorrente` costruito dal `middleware.php` (che continua a esporre `$user_info`, `$u_id`, `$is_full_admin`, `$is_gestore` per le pagine non migrate); `ServizioPermessi` per `ha_modulo`, `utente_ha_abilitazioni`, `utente_gestisce_didattica`, `require_admin_or_gestore`; `MetadatiAccesso` (SPID/CIE/Ateneo) per `auth_meta`.
- `saml_login.php` e `sync_sso_user()`: il codice SimpleSAML **si sposta senza cambiare logica** in `SincronizzazioneSso`, con prove sul percorso locale `__accesso` e controllo manuale del login reale in un orario di basso traffico.
- CSRF (`csrf_field`/`csrf_verify`, `inc/sistema.php`) e rate limit (`check_rate_limit`, `inc/base.php`) → `Core\Csrf`, `Core\RateLimiter`.
- **Perché dopo il pilota:** è il punto più critico del sito; ci si arriva con Core, infrastruttura e procedura già collaudati.

STEP 6 — Portale: testata, menu, home, organizzazione proposta

- `inc/sezioni.php`, `aspetto.php`, `organizzazione.php` → `Portale\*`; `header.php` diventa un presenter (la recente regressione del menu dovuta a una variabile `$m` riusata nell'header è esattamente il tipo di errore che le variabili locali delle classi eliminano).
- Prova da aggiungere prima: confronto dell'HTML del menu per utente anonimo e amministratore.

STEP 7 — Anagrafi e catalogo di Ateneo (sola lettura, molto usati)

- `anagrafi.php`, `catalogo_ateneo.php`, `cerca_insegnamenti.php`, `cerca_scuole.php`, `cerca_personale.php`: `Repository` e `ClientApiAteneo` (chiamate HTTP alle API di Ateneo isolate e simulabili nei test).
- Servono a Eventi, FSL, Tutorato e Didattica: migrarli presto semplifica tutto il resto.

STEP 8 — Eventi: aree, eventi, turni, agenda, seminari, report per ambito

- `eventi_progetti.php`, `agenda.php`, `seminari.php`, `report_ambiti.php`, pagine `area.php`, `agenda.php`, `agenda_ics.php`, schede admin eventi/progetti.

STEP 9 — Iscrizioni: prenotazioni, liste d'attesa, check-in, attestati, sondaggi

- Cuore del portale e unico punto con concorrenza (`getPostiOccupati(..., FOR UPDATE)`): `ServizioPrenotazioni::prenota()` dentro `Database::transazione()`, con prova di concorrenza (due prenotazioni parallele sull'ultimo posto) da aggiungere prima di migrare.
- Cron (`cron_attestati.php`, `cron_background.php`, `admin/cron_reminders.php`) come comandi che chiamano i Service.

STEP 10 — Risorse, aule e sportelli di ricevimento

- `risorse.php`, `calendario_risorse.php`, `prenotazioni_risorse`, `ricevimento.php`, `risorsa_ics.php`.

STEP 11 — FSL

- `fsl.php` (convenzioni compilate online, documenti Word, firme PAdES, valutazioni).

STEP 12 — Tutorato

- `tutorato.php`, `tutorato_registro.php`, `incarico.php`, `firma_incarico.php`, `registro_tutorato.php`: `ServizioFirmePades` condiviso poi da Sedute (verbali) — firma remota Aruba isolata dietro un'interfaccia `FirmaRemota` (simulabile nei test).

STEP 13 — Didattica: uffici, moduli, pratiche, iter, protocollo, convalide, segreteria

- Il modulo più grande (`didattica.php` 80 funzioni + `convalide.php` 20) e il più ricco di regole. Si divide in sotto-step rilasciabili:
- 13a Uffici e Operatori (+ `TipoUfficio`, risoluzione ufficio del corso);
- 13b Modulo e campi (costruttore, logica condizionale, colonne tipizzate, lettura delle risposte: oggi `campi_modulo`, `leggi_risposte_modulo`);
- 13c Pratica e iter (`crea_pratica`, `assegna_pratica`, stati, eventi, email);
- 13d Protocollo, marca da bollo, domanda PDF/A (`GeneratoreDomandaPdfa`), invio alla segreteria;
- 13e Convalide e piano di studi (`DecisioniConvalida`, frasi per verbale ed estratto).
- Le schede `admin/didattica_*.php` diventano controller sottili; `modulo.php`, `pratiche.php`, `allegato_pratica.php` idem.
- Il flusso protocollo → ufficio del corso → segreteria diventa configurazione del `Modulo` gestita dal `ServizioIter`: è la base generica per tutte le pratiche future richieste.

STEP 14 — Sedute, consigli e verbali

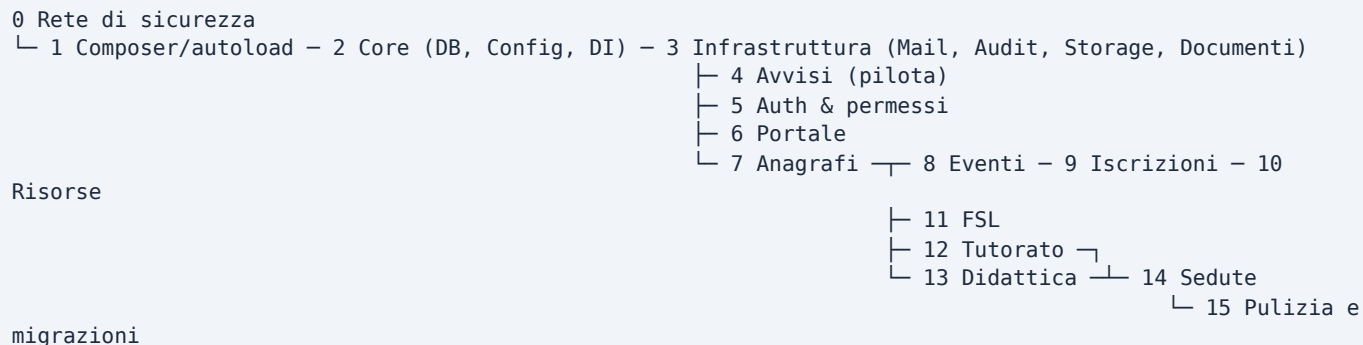
- `sedute.php` e parte di `didattica.php` (consigli, presenze, convocazioni, decisioni in seduta, verbale Word, estratti PDF/A, firma PAdES segretario → coordinatore).

STEP 15 — Schema, cron, pulizia e (facoltativo) front controller

- `assicura_schema()` → `App\Core\Migrazioni` con una classe per versione (v44, v45...), stesso marcatore in `cache/`, stessa idempotenza.
- Eliminazione delle facciate non più chiamate, di `App\Core\App` (ponte statico) dove possibile, delle costanti array sostituite dagli enum.
- Facoltativo: `public/index.php` con router e template (Twig o PHP puro) mantenendo gli stessi URL tramite le regole del `.htaccess` esistente; si valuta solo quando tutte le pagine sono già controller sottili.

- PHPStan portato al livello 8 su tutto `src/`.

Ordine e dipendenze in sintesi



Gli step 8-14 dipendono solo da 1-7 e possono essere riordinati in base alle priorità del Dipartimento (es. Didattica prima di Eventi se arrivano nuove pratiche da gestire).

Criteri di "step completato"

- Tutte le prove esistenti verdi + nuovi test PHPUnit del modulo verdi.
- PHPStan senza errori sul nuovo codice al livello concordato.
- Nessuna query SQL nelle pagine del modulo né nei Service (solo nei Repository).
- Funzioni legacy del modulo ridotte a facciate (o rimosse se senza chiamanti).
- Pagine e PDF verificati a mano in locale; elenco file di rilascio pronto.
- Dopo l'upload: `prove_server.sh` e `verifica_sito.sh` verdi.

Rischi principali e contromisure

Rischio	Contromisura
<code>vendor/</code> mancante o incompleto in produzione	Aggancio condizionale (1.4) nel primo rilascio; dal secondo step <code>prove_server.sh</code> controlla la presenza dell'autoload
Differenze sottili di comportamento (tipi restituiti da mysql, stringhe vs interi, errori silenziosi)	Stesso driver mysql; facciate con risultato identico; prove di caratterizzazione prima di ogni step
Login SSO/SAML rotto	Step 5 dopo il pilota, logica spostata senza modifiche, prova reale in orario di basso traffico, rollback immediato dei soli file dello step
Overbooking delle prenotazioni	Transazione e <code>FOR UPDATE</code> conservati in <code>ServizioPrenotazioni</code> ; prova di concorrenza dedicata
Lavoro in parallelo con nuove funzionalità	Step brevi che vanno in <code>main</code> appena verdi; nuove funzionalità scritte direttamente come classi in <code>src/</code> per i moduli già migrati
Deriva del piano	Inventario automatico (Step 0) aggiornato a ogni step: numero di funzioni procedurali rimaste per modulo